

SE-12-02: Testing and Evaluation Methodologies

In the **NSW Software Engineering 11–12 Syllabus (2022)**, testing is a systematic process used to verify that a software solution functions as intended and meets the needs of the stakeholder. For a Band 6 project, you must demonstrate a rigorous approach by applying specific methodologies to identify logic errors and validate system performance.

1. Systematic Testing Methodologies

Testing should not be ad-hoc; it must follow a structured plan that covers all potential execution paths and data limits.

Path Testing

This methodology ensures that every possible logical path through the code is executed at least once.

- **Purpose:** To identify "dead code" or logic errors in conditional statements (e.g., if/else blocks).
- **Application:** In your BMI calculator, path testing involves entering data that triggers every category: Underweight, Healthy, Overweight, and Obese.

Boundary Testing

This focuses on the "edges" of input ranges where errors are most likely to occur.

- **Purpose:** To ensure the system handles transition points correctly (e.g., exactly 18.5 or 25.0 BMI).
- **Application:** Testing a height of 0 cm or a weight of 500 kg to ensure the system handles extreme or invalid values without crashing.

2. Creating a Test Plan (Component B Requirement)

A comprehensive test plan documents the intended testing process before execution. It must include:

- **Test Cases:** Specific scenarios to be tested.
- **Test Data:** The actual values (normal, boundary, and erroneous) entered into the system.
- **Expected vs. Actual Outputs:** A comparison used to determine if the test passed or failed.

Example Test Plan Table

Test ID	Description	Input Data	Expected Output	Actual Output	Result
01	Normal BMI	H: 180, W: 75	23.15 (Healthy)	23.15 (Healthy)	Pass +1
02	Boundary (Low)	H: 180, W: 60	18.52 (Healthy)	18.52 (Healthy)	Pass +1
03	Invalid Input	H: "abc", W: 70	Error Message	Error Message	Pass +1

3. Evaluation and Refinement

Evaluation is the process of judging the effectiveness of the final solution against the original requirements.

- **Code Optimisation:** After testing, apply language-dependent techniques (e.g., refactoring Python loops or using more efficient data structures) to improve performance.
- **Feedback Synthesis:** Collect and analyse feedback from peers, teachers, and clients. This feedback must be synthesised into a formal report that explains how the code was refined to improve quality.
- **Final Effectiveness:** Critically assess how well the software solves the original problem and identify any remaining limitations for future development.

Why It Matters (Rationale)

Without rigorous testing, software is prone to failure in real-world scenarios. For an engineer, a formal evaluation provides "traceability," proving that every functional requirement listed at the start of the project has been successfully implemented and verified.

Common Mistakes

- **Only Testing "Happy Paths":** Only using data that you know will work, rather than trying to "break" the system with invalid inputs.
- **Vague Results:** Writing "it works" in the actual output column instead of the specific value or message returned by the program.

6-Mark HSC Model Response

Question: *Discuss how the application of path and boundary testing contributes to the quality assurance and evaluation of a software solution.*

Model Answer: The application of systematic testing methodologies is critical for ensuring a software solution is robust and meets all functional requirements. **Path testing** ensures quality by verifying every logical branch within the code. For example, in a BMI calculator, it confirms that the algorithm correctly navigates the if-elif-else structure to assign the correct category to a user. By executing every path, the developer can ensure there is no unreachable code, which improves the overall reliability of the system.

Complementing this, **boundary testing** evaluates how the system handles extreme or transition values. By testing data at the exact limits of a range—such as the specific BMI value that separates 'Healthy' from 'Overweight'—the developer ensures the mathematical logic is precise. This methodology also involves entering erroneous data, such as negative numbers or strings, to verify that **error handling** routines prevent system crashes. Together, these methodologies allow for a thorough **evaluation** where expected and actual outputs are compared to provide documented evidence that the software is functional, secure, and ready for stakeholder use.